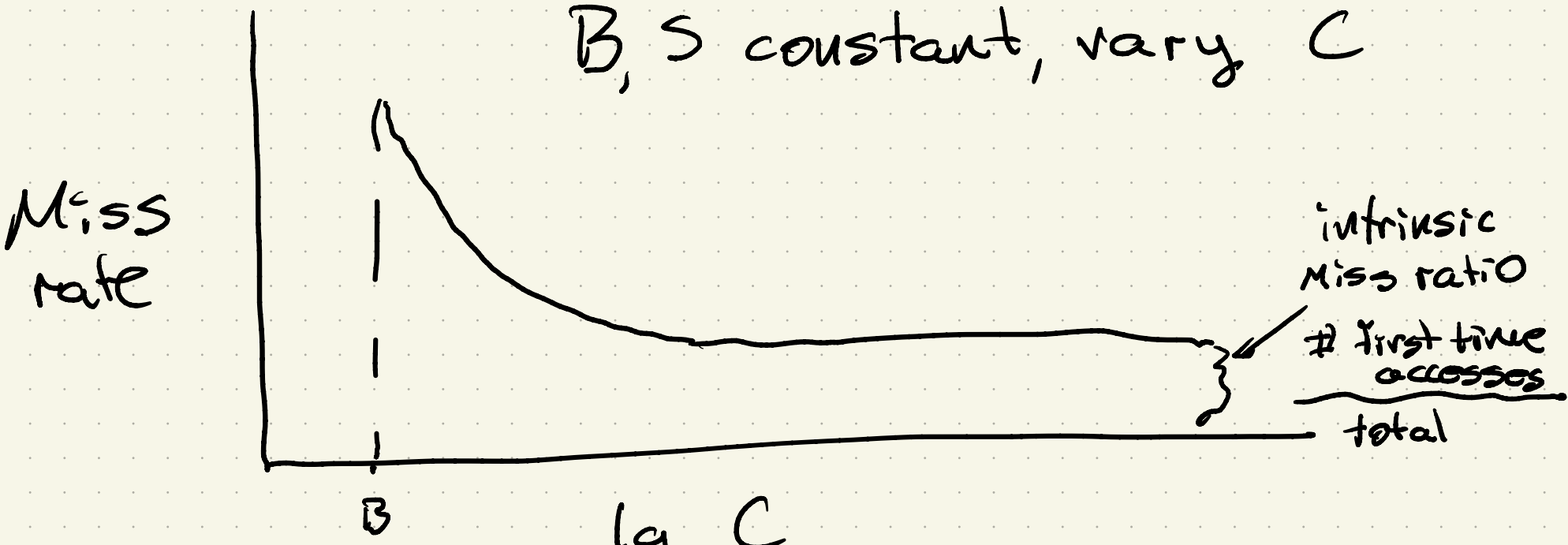
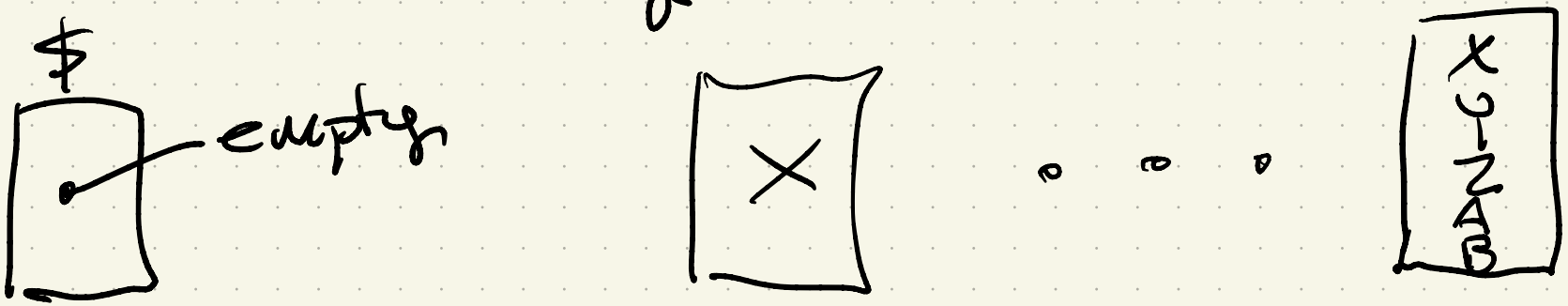
Impact of changing C, B, S

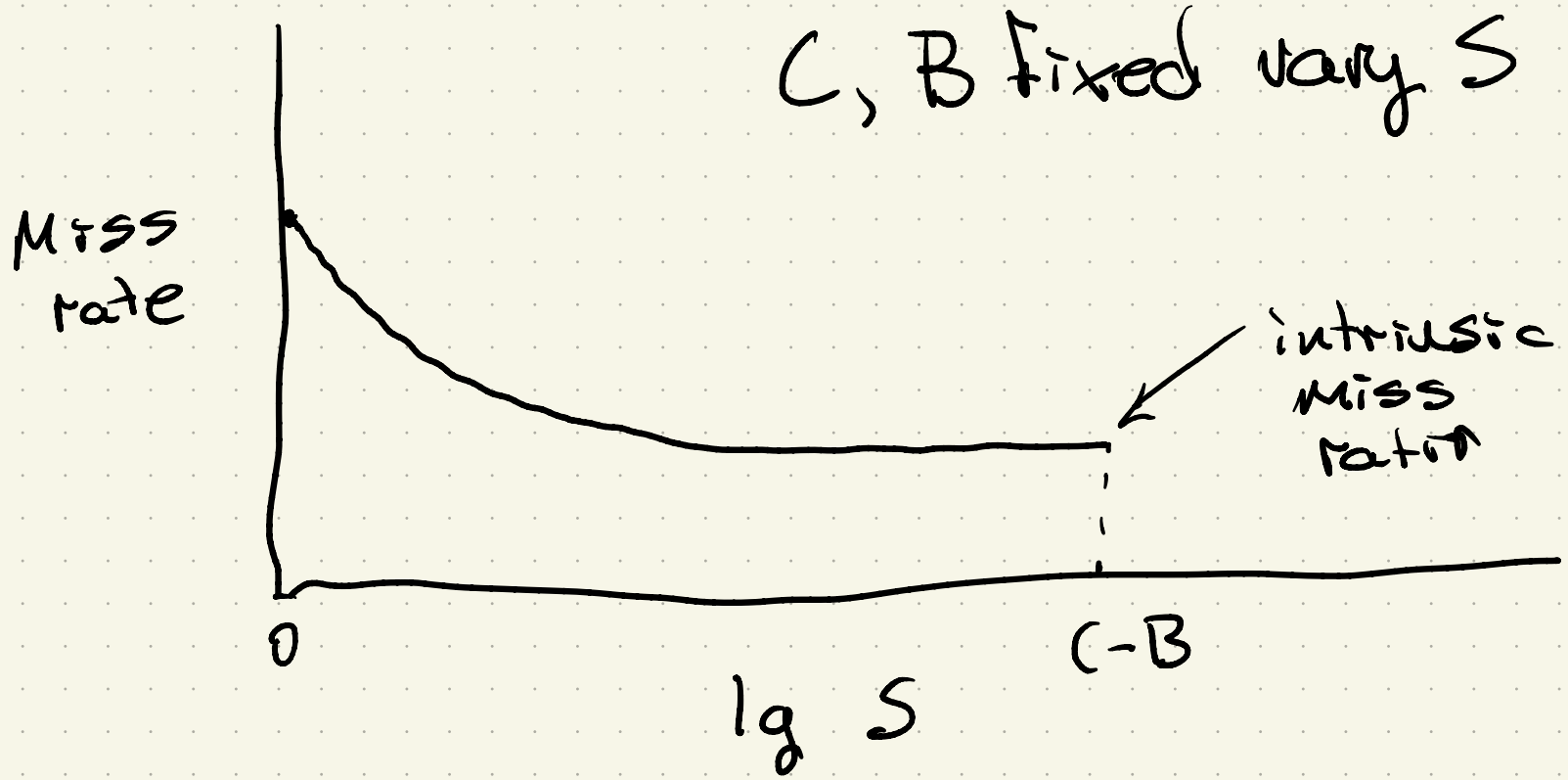


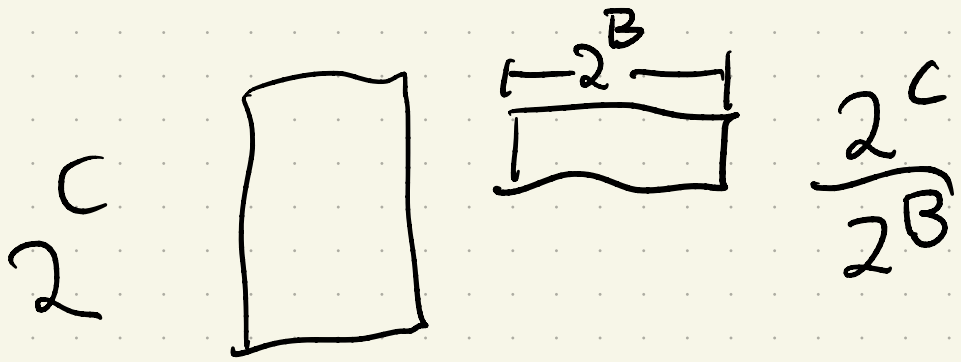


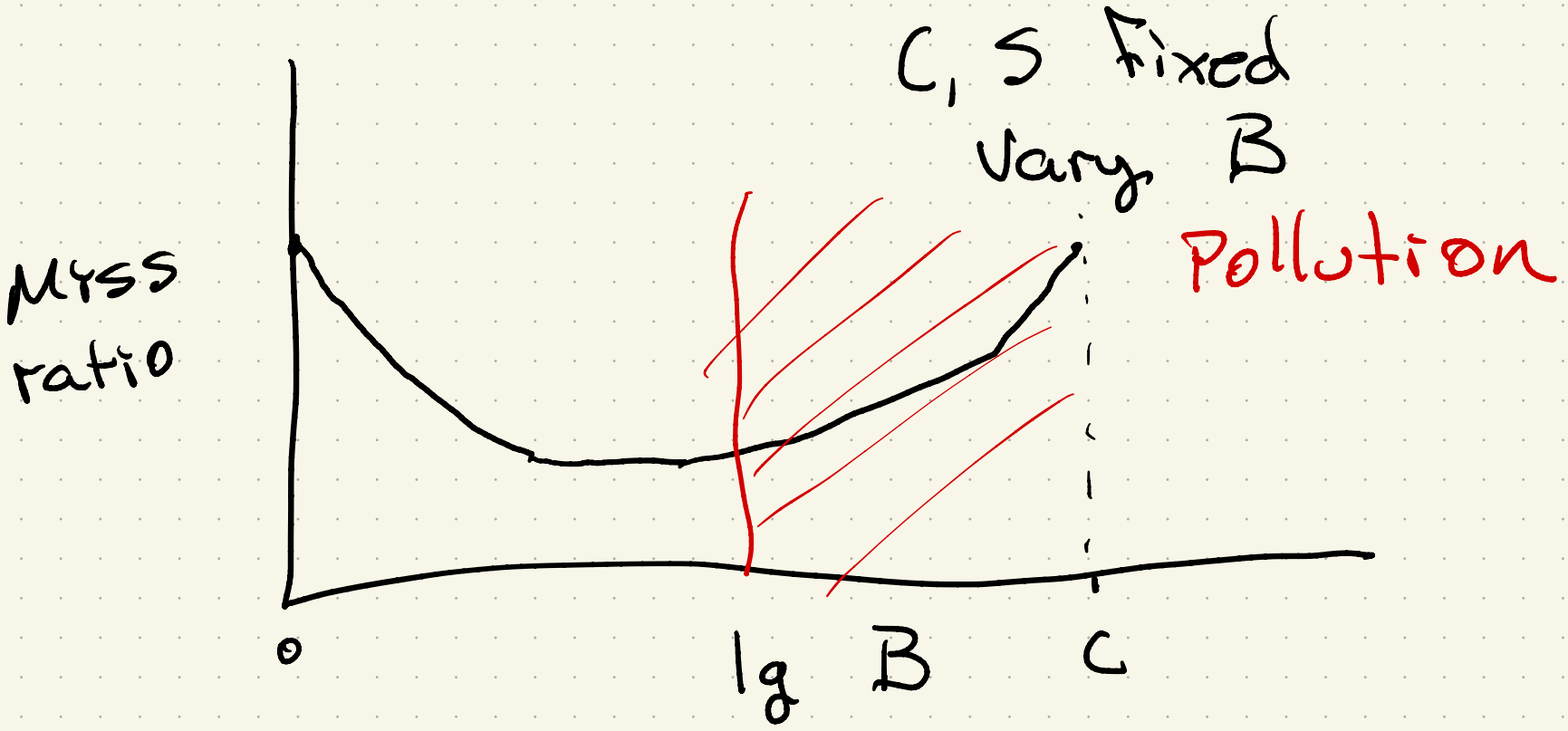
start access X Y Z A B . . C

Demand fetching

compulsory miss


C, B fixed vary S
Miss rate
intrinsic miss rate
0
C-B
lg S


2^C
2^B
$\frac{2^C}{2^B}$


C, S Fixed
Vary B
Pollution
Mass ratio
0
lg B
C

Write strategies

CPU writes to address x

x is in $

Write to cache block for x

Also update memory?

Yes: Write through (WT)

No: just update $

When is memory updated?

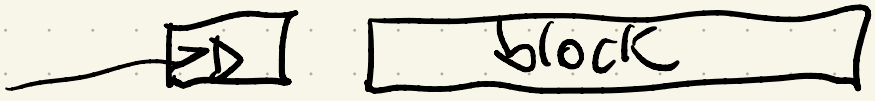
When blocks are replaced

Write back (WB)

opt Only write back blocks that have been written to

Dirty bit

status info



D ← 1 on writes

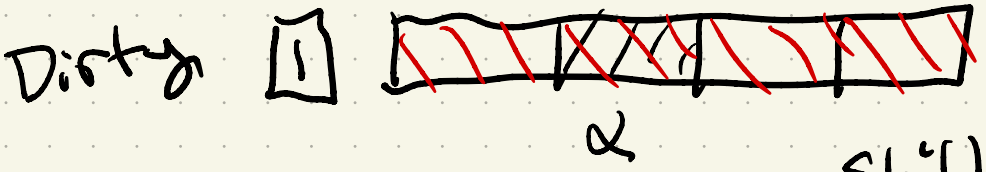
if D=1 on replacement then write back

X is not in $

if doing write back, then
must allocate space in $ for
block (write (miss) allocate)(WA)

WB WA

X is a write to a byte



Still a miss
Still get whole block
overwrite X

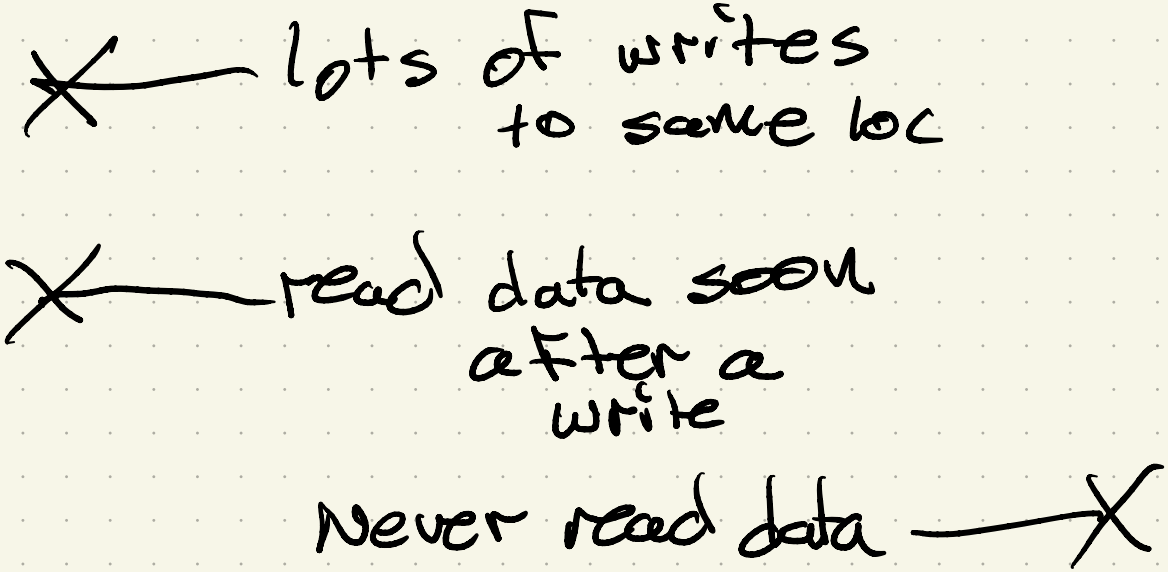
WT WNA

if doing write thru, then
Don't allocate, just update
memory
Write (miss) no allocate (WNA)

which one?

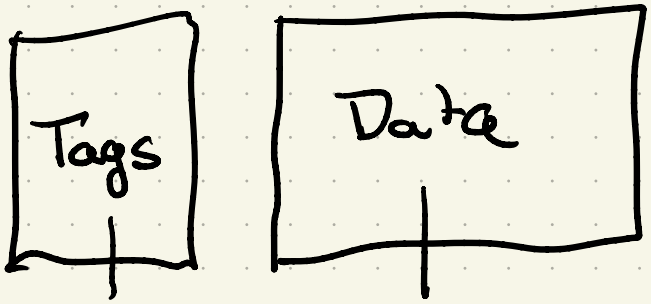
WB WA or WTWNA

Program behavior:



Two cache design approaches

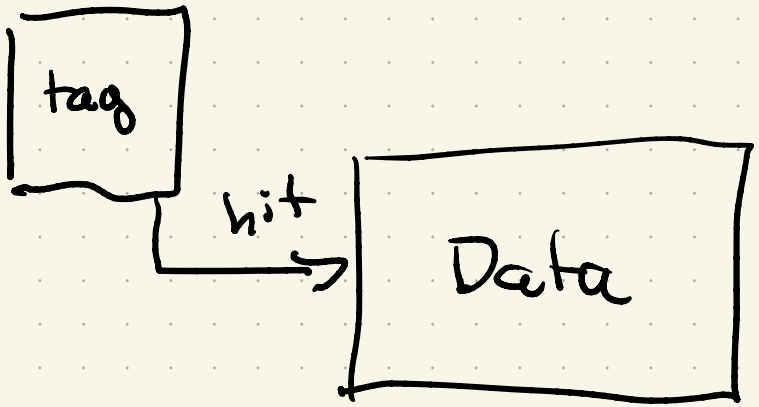
✓



tag check after set (data) access

AAT = HT + MR × MP

✗



Tag check first access data on hit

AAT = (1-MR)·HT + MR·MP

Virtual memory - what's that?

OS software implemented $

$ = DRAM, fully associative

Memory = swap space on disk

Terms:

Virtual memory	$
Page number	Tag
Page	Block
Page offset	offset
Page Fault	Miss
"inverted page table"	Tag store
Page table	n/a
n/a	index
Page Frame	"way"

Back to caches ...

Types of misses

1) Compulsory misses
   First time miss to data

2) Capacity misses
   In fully assoc $, run out of space → causes subsequent miss

3) Conflict miss
   Direct-mapped $ two (or more) frequent accesses with the same index (= set number in set-assoc.)

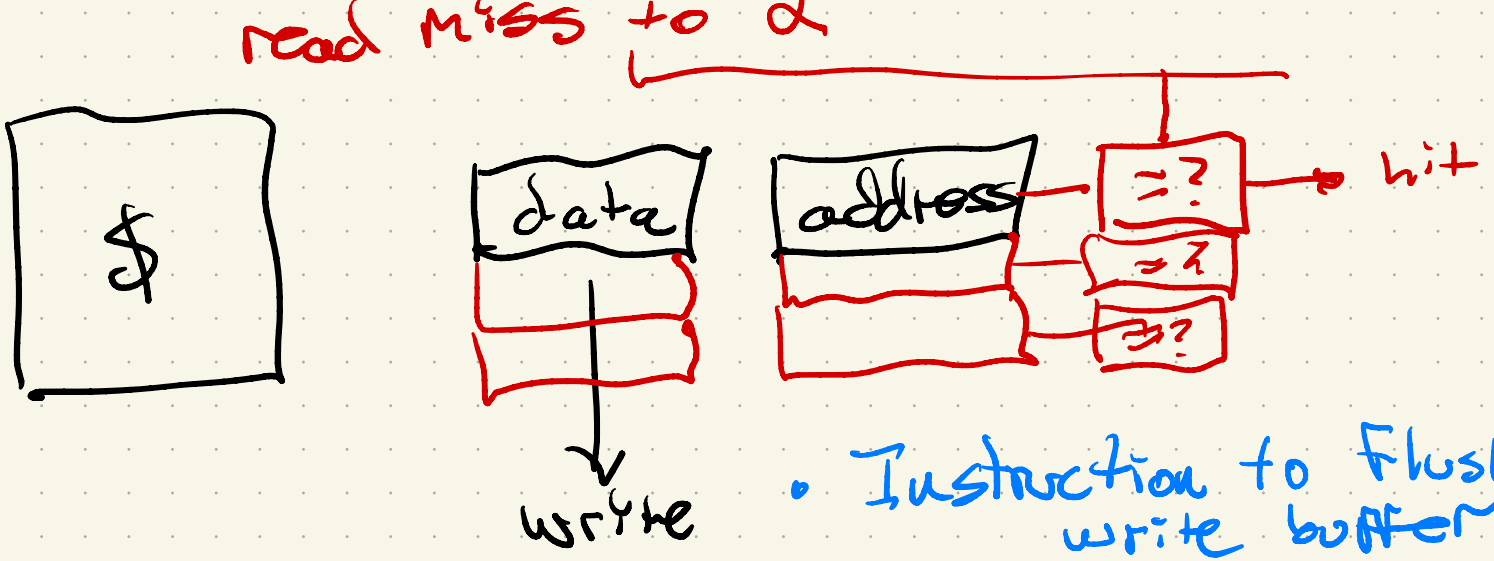
conflict miss if occur in DM/S.A. $ that would not occur in the same sized F.A. $

Improve $ performance?

$$AAT = \text{Hit time} + (\text{Miss rate}) \times (\text{Miss penalty})$$

Miss penalty reduction techniques:

1) Dont wait For writes to complete



Write buffer

- Instruction to flush write buffer
- Instruction to disable write buffer